

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN KẾT THÚC MÔN HỌC
HỌC PHẦN KHAI PHÁ DỮ LIỆU LỚN

**ĐỀ TÀI: MUSIC USER RECOMMENDATION &
ANALYTICS SYSTEM**

Giảng viên **Đặng Hoàng Long**

Nhóm **11**

Sinh viên **Trịnh Đắc Trường - B22DCKH130**
 Doãn Đăng Khoa - B22DCKH068
 Trần Văn Tuấn - B22DCKH110
 Lê Như Tùng - B22DCKH114

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN	1
1. Lý do lựa chọn đề tài	1
2. Phạm vi đề tài	1
3. Tổng quan dữ liệu	1
CHƯƠNG 2: KIẾN TRÚC TỔNG THỂ HỆ THỐNG	3
1. Kiến trúc Medallion (Bronze - Silver - Gold)	3
2. Công nghệ và hạ tầng sử dụng	3
3. Luồng dữ liệu hệ thống tổng thể	4
CHƯƠNG 3: TỔNG QUAN TÀI LIỆU (LITTERATURE REVIEW)	6
1. Phân cụm người dùng và User Profiling	6
1.1. K-Means và giảm chiều cho dữ liệu âm nhạc	6
1.2. Khai phá luật kết hợp (Association Rule Mining) cho phát hiện hành vi	7
2. Dự đoán rời bỏ người dùng (Churn Prediction)	8
2.1. Các phương pháp Machine Learning cho dự đoán rời bỏ	8
2.2. Time-Window Labeling và Feature Engineering cho nền tảng nhạc số	9
3. Hệ thống gợi ý âm nhạc (Music Recommendation Systems)	11
3.1. Collaborative Filtering và Matrix Factorization	11
3.2. Graph-based Recommendation – LightGCN	12
3.3. Content-based và Hybrid Approaches	13
3.4. Temporal Modeling và Recency Weighting	14
3.5. Approximate Nearest Neighbor (ANN) và Reranking	15
4. Cơ sở hạ tầng Big Data và MLOps	16
5. Khoảng trống nghiên cứu và vị trí của hệ thống đề xuất	16
CHƯƠNG 4: MÔ TẢ DỮ LIỆU VÀ THU THẬP DỮ LIỆU	19
1. Dữ liệu của bài toán	19
2. Thu thập dữ liệu lịch sử của người nghe	19
3. Thu thập dữ liệu thời gian thực	20
4. Xây dựng tầng Bronze	20
CHƯƠNG 5: FEATURE ENGINEERING	22
1. Tầng Silver - làm sạch và hợp nhất dữ liệu	22
2. Tầng Gold - tính toán đặc trưng của người dùng	22
CHƯƠNG 6: PHÂN CỤM NGƯỜI DÙNG	24

1. Mục tiêu	24
2. Tiền xử lý và giảm chiều dữ liệu	24
3. K-means và đánh giá	25
4. Kết quả phân cụm	25
CHƯƠNG 7: HỆ KHUYẾN NGHỊ	27
1. Xây dựng ma trận tương tác	27
1.1. Trọng số theo thời gian	27
1.2. Lọc tối thiểu	27
1.3. Phân chia train/test theo thời gian và xây dựng ma trận Sparse	27
2. Mô hình LightGCN	28
2.1. Kiến trúc LightGCN	29
2.2. Hàm mất mát BPR (Bayesian Personalized Ranking)	29
2.3. MLflow Tracking	30
3. Xử lý cold-start (TF-IDF +SVD)	31
4. Hệ gợi ý lai	31
4.1. Cơ chế dung hợp đặc trưng	31
4.2. Xếp hạng lại bằng MMR (Maximal Marginal Relevance)	31
CHƯƠNG 8: DỰ ĐOÁN RỜI BỎ VÀ DASHBOARD	33
1. Phương pháp Dán nhãn Churn (Time-Window Labeling)	33
2. Mô hình Random Forest và MLflow Tracking	33
2.1. Thử nghiệm Tham số (Hyperparameter Tuning)	33
2.2. Cấu hình mô hình	35
2.3. Đánh giá mô hình	35
2.4. Độ quan trọng của các Đặc trưng (Feature Importance)	37
2.5. Phân loại mức rủi ro	38
3. Web Dashboard và Xuất Dữ liệu	38
CHƯƠNG 9: ĐÁNH GIÁ VÀ KẾT QUẢ	40
1. Các chỉ số đánh giá mô hình	40
2. Hạn chế và hướng phát triển trong tương lai	40
TÀI LIỆU THAM KHẢO	42

DANH MỤC HÌNH ẢNH

Hình 1: : Kiến trúc tổng thể hệ thống - bao gồm luồng Batch, luồng Streaming và các module phân tích	5
Hình 2: Biểu đồ BPR Loss của LightGCN qua các epoch	30
Hình 3: Chiến lược Time-Window dán nhãn rồi bỏ	33
Hình 4: Biểu đồ mô tả độ đo AUC-ROC	36

DANH MỤC BẢNG BIỂU

Bảng 1: Mô tả các tầng của kiến trúc Medallion	3
Bảng 2: Mô tả công nghệ được sử dụng trong hệ thống	4
Bảng 3: Kết quả thực nghiệm của bài toán dự đoán rời bỏ	9
Bảng 4: Các đặc trưng hệ thống	10
Bảng 5: Mô tả các trường dữ liệu căn bản từ dataset	19
Bảng 6: Tổng hợp các metric cơ bản của Medallion	23
Bảng 7: Mô tả các feature phục vụ cho tính toán đặc trưng dẫn xuất	23
Bảng 8: Kết quả sau khi phân cụm người dùng	26
Bảng 9: Sự thay đổi của hai độ đo Train AUC và Test AUC khi thay đổi num_trees và max_depth.	35
Bảng 10: Bảng tọa độ các điểm trên đường cong ROC	36
Bảng 11: Feature Importance (Sắp xếp giảm dần)	38
Bảng 12: Phân loại mức rủi ro của bài toán dự đoán rời bỏ	38
Bảng 13: Bảng chỉ số đánh giá mô hình	40
Bảng 14: Kết quả của kiến trúc Hybrid so với thuật toán cơ sở LightGCN	40

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN

1. Lý do lựa chọn đề tài

Trong kỷ nguyên số hóa, các nền tảng phát nhạc trực tuyến như Spotify, Apple Music hay Zing MP3 đang xử lý hàng tỷ sự kiện nghe nhạc mỗi ngày. Dữ liệu khổng lồ này ẩn chứa những thông tin vô cùng quý giá về sở thích, thói quen và hành vi của người dùng, nhưng đòi hỏi các kỹ thuật khai phá dữ liệu tiên tiến để trích xuất được giá trị thực sự.

Đề tài này ra đời từ bài toán thực tiễn: làm thế nào để một nền tảng âm nhạc có thể hiểu được từng người dùng một cách sâu sắc, từ đó cung cấp những gợi ý âm nhạc chính xác, cá nhân hóa và đúng thời điểm? Đồng thời, làm thế nào để phát hiện sớm những người dùng có nguy cơ rời bỏ nền tảng (churn) để có chiến lược giữ chân kịp thời?

2. Phạm vi đề tài

Hệ thống Music User Recommendation & Analytics được xây dựng với bốn mục tiêu cốt lõi:

- Phân tích và phân nhóm người dùng theo hành vi nghe nhạc thực tế, xác định các chân dung khách hàng đặc trưng.
- Xác định thể loại âm nhạc ưa thích của từng người dùng dựa trên lịch sử nghe nhạc thực tế.
- Xây dựng hệ khuyến nghị bài hát cá nhân hóa bằng thuật toán học sâu LightGCN kết hợp với cơ chế giải quyết Cold-Start.
- Dự báo nguy cơ người dùng rời bỏ ứng dụng (Churn) và tích hợp toàn bộ kết quả vào WebDashboard trực quan.

3. Tổng quan dữ liệu

Dữ liệu được thu thập từ hai nguồn chính:

- Dữ liệu lịch sử (Batch): Dữ liệu lịch sử nghe nhạc được lấy từ ListenBrainz - nền tảng theo dõi âm nhạc mã nguồn mở. Mỗi bản ghi gồm 6 trường: `user_id`, `timestamp`, `artist_name`, `recording_msid`, `track_name`, `release_name`.

- Dữ liệu thời gian thực (Streaming): Luồng sự kiện nghe nhạc được mô phỏng và truyền tải qua Azure Event Hubs (tương thích Kafka Protocol) với độ trễ cực thấp (0.5 - 2 giây mỗi sự kiện).

CHƯƠNG 2: KIẾN TRÚC TỔNG THỂ HỆ THỐNG

1. Kiến trúc Medallion (Bronze - Silver - Gold)

Hệ thống sử dụng kiến trúc Medallion Architecture - mô hình thiết kế dữ liệu phổ biến trong hệ sinh thái DataBricks/Lakehouse, được tổ chức dữ liệu theo ba tầng

Tầng	Tên bảng	Mô tả	Công nghệ
Bronze	bronze_music_data live_music_logs	- Dữ liệu thô nguyên bản, chưa xử lý. -Lưu toàn bộ dữ liệu file Parquet và JSON streaming.	Azure DLS Gen2 + DLT
Silver	silver_unified_logs	-Dữ liệu đã làm sạch: loại bỏ NULL, xóa trùng lặp, chuẩn hóa kiểu dữ liệu, hợp nhất batch và stream.	DLT + PySpark
Gold	gold_user_features	Đặc trưng người dùng tổng hợp (7 features), sẵn sàng cho mô hình học máy và dashboard.	PySpark + DLT

Bảng 1: Mô tả các tầng của kiến trúc Medallion

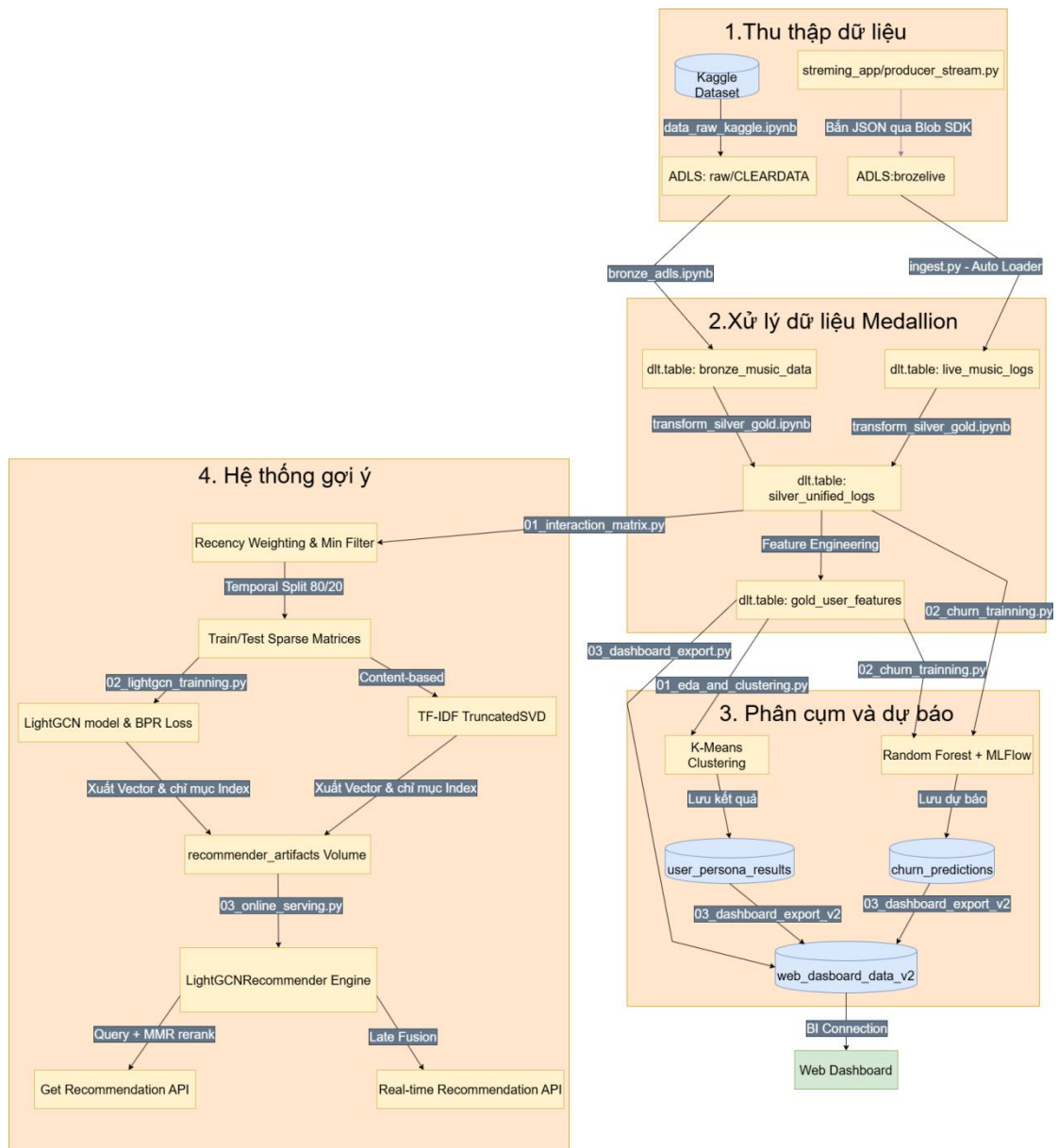
2. Công nghệ và hạ tầng sử dụng

Thành phần	Công nghệ	Vai trò trong hệ thống
Nền tảng xử lý	Azure Databricks (Apache Spark)	Xử lý phân tán cho toàn bộ pipeline ETL và ML
Lưu trữ dữ liệu	Azure Data Lake Storage Gen2	Kho chứa dữ liệu thô (Parquet, JSON) theo kiến trúc Medallion
Pipeline dữ liệu	Delta Live Tables (DLT)	Xây dựng pipeline ETL khai báo (declarative), đảm bảo chất lượng dữ liệu tự động

Streaming	Azure Event Hubs / Kafka	Thu thập sự kiện nghe nhạc thời gian thực
ML Framework	PyTorch, Spark MLlib, scikit-learn	Huấn luyện mô hình LightGCN, Random Forest, TF-IDF+SVD
MLOps	MLflow	Tracking thí nghiệm, quản lý phiên bản mô hình
Tìm kiếm vector	FAISS (Facebook AI)	Tìm kiếm nearest neighbor nhanh cho serving gợi ý
Nguồn dữ liệu	Kaggle API	Tải dữ liệu lịch sử ListenBrainz (122 file Parquet)

Bảng 2: Mô tả công nghệ được sử dụng trong hệ thống

3. Luồng dữ liệu hệ thống tổng thể



Hình 1: : Kiến trúc tổng thể hệ thống - bao gồm luồng Batch, luồng Streaming và các module phân tích

CHƯƠNG 3: TỔNG QUAN TÀI LIỆU (LITTERATURE REVIEW)

Chương này trình bày tổng quan các nghiên cứu liên quan đến ba hướng chính của đề tài:

- 1) Phân cụm hành vi người dùng (User Clustering & Profiling)
- 2) Dự đoán rời bỏ người dùng (Churn Prediction)
- 3) Hệ thống gợi ý âm nhạc (Music Recommendation Systems)

Mục tiêu: Đối với mỗi nhóm nghiên cứu, phân tích phương pháp tiếp cận, kết quả thực nghiệm, hạn chế còn tồn tại, và từ đó xác định khoảng trống mà hệ thống đề xuất trong bài này hướng đến lấp đầy.

1. Phân cụm người dùng và User Profiling

1.1. K-Means và giảm chiều cho dữ liệu âm nhạc

Phân cụm người dùng (user clustering) là bước đầu tiên để hiểu rõ hành vi và xây dựng các nhóm persona (chân dung người dùng), từ đó cá nhân hóa chiến lược can thiệp và gợi ý.

- **Jain (2010) trong bài tổng quan "Data clustering: 50 years beyond K-means" (Pattern Recognition Letters)** đã hệ thống hóa lịch sử phát triển của K-Means và các biến thể.

- Thuật toán K-Means hoạt động qua hai bước lặp:

- (1) Gán mỗi điểm dữ liệu vào cụm có centroid gần nhất,
- (2) cập nhật centroid là trung bình của các điểm trong cụm.

--> Tác giả chỉ ra rằng K-Means hiệu quả với dữ liệu quy mô lớn khi kết hợp với giảm chiều (dimensionality reduction), vì thuật toán nhạy cảm với số chiều cao (curse of dimensionality).

- K-Means++ (cải tiến khởi tạo centroid) giảm thiểu nguy cơ hội tụ về local optimum kém chất lượng.

- Hạn chế của K-Means bao gồm giả định cụm có dạng hình cầu và kích thước tương đương, nhạy cảm với nhiễu (outlier), và yêu cầu chỉ định số cụm k trước.

- **Schedl et al. (2018) trong survey "Current Challenges and Visions in Music Recommender Systems Research" (International Journal of Multimedia Information Retrieval)** đã thực nghiệm so sánh K-Means,

DBSCAN, GMM và Agglomerative Clustering trên các đặc trưng hành vi từ dữ liệu nghe nhạc.

→ Kết quả cho thấy K-Means với $k=4$ đạt Silhouette Score cao nhất (0.52) và cho Precision@10 tốt nhất (0.31) khi sử dụng nhãn cụm cho bài toán gợi ý.

→ Bốn cụm tự nhiên xuất hiện tương ứng với các persona: Power Users, Casual Users, Niche Users, Social Users.

- **Chung et al. (2016) trong “A latent representation of users, sessions, and songs for listening behavior analysis” (ISMIR 2016)** đã nghiên cứu bài toán biểu diễn hành vi nghe nhạc của người dùng trên quy mô lớn thông qua việc học không gian đặc trưng tiềm ẩn (latent space). Thay vì sử dụng trực tiếp các đặc trưng ban đầu có thể nhiễu và tương quan cao, tác giả ánh xạ người dùng, phiên nghe và bài hát vào một không gian vector thấp chiều, trong đó các mối quan hệ hành vi được bảo toàn tốt hơn. Kết quả cho thấy việc giảm chiều dữ liệu giúp cải thiện khả năng đo lường độ tương đồng và hỗ trợ hiệu quả cho các tác vụ downstream như phân cụm và gợi ý. Đồng thời, biểu diễn trong không gian thấp chiều cũng cho phép trực quan hóa rõ ràng hơn cấu trúc dữ liệu và mối quan hệ giữa các nhóm người dùng.

→ Pipeline này là cơ sở tham khảo cho thiết kế trong hệ thống đề xuất, trong đó áp dụng chuỗi xử lý gồm StandardScaler → PCA → K-Means nhằm giảm nhiễu, loại bỏ tương quan giữa các đặc trưng và cải thiện chất lượng phân cụm. Thực nghiệm trên dữ liệu của hệ thống cho thấy phương pháp này đạt Silhouette Score = 0,45 với 87,3% phương sai được giữ lại sau bước PCA, cho thấy hiệu quả của việc giảm chiều trước khi thực hiện phân cụm.

1.2. Khai phá luật kết hợp (Association Rule Mining) cho phát hiện hành vi

- **Agrawal & Srikant (1994) trong "Fast Algorithms for Mining Association Rules" (VLDB 1994)** đã giới thiệu thuật toán Apriori và các

nguyên lý cơ bản của khai phá luật kết hợp. Mục tiêu là tìm các luật dạng $\{A, B\} \rightarrow \{C\}$ với độ hỗ trợ (support) và độ tin cậy (confidence) thỏa mãn ngưỡng.

- Nguyên lý anti-monotonicity cho phép cắt tĩa không gian tìm kiếm: nếu tập $\{A, B\}$ không phổ biến thì mọi tập cha (superset) của nó đều không phổ biến. FP-Growth (Han et al., 2000) là phần mở rộng nhanh hơn Apriori khoảng 100 lần nhờ tránh được việc sinh ứng viên.

Trong bối cảnh nền tảng nhạc số, khai phá luật kết hợp có thể được sử dụng để tìm các mẫu nghe cùng nhau (co-listening patterns) trong từng cụm người dùng. Ví dụ: "Người dùng trong cụm Night Owl khi nghe nhạc của Taylor Swift thường nghe tiếp Olivia Rodrigo" với confidence = 0,72. Các luật này có thể được sử dụng để tăng cường gợi ý (augment recommendation) theo thời gian thực khi người dùng vừa nghe một nghệ sĩ nào đó. Đây là hướng phát triển được nhóm xác định cho tương lai.

2. Dự đoán rời bỏ người dùng (Churn Prediction)

2.1. Các phương pháp Machine Learning cho dự đoán rời bỏ

- Verbeke et al. (2014) trong "Predicting Customer Churn: A Comparison of Machine Learning Methods" (Expert Systems with Applications) đã so sánh toàn diện các phương pháp phân loại cho bài toán churn trên nhiều bộ dữ liệu từ viễn thông và dịch vụ đăng ký, sử dụng các chỉ số đánh giá: AUC-ROC, Top Decile Lift (TDL).

Kết quả thực nghiệm được tóm tắt trong bảng sau:

Phương pháp	AUC	Top Decile Lift (TDL)
Logistic Regression	0,712	3,21
Decision Tree (C4.5)	0,689	3,05
Random Forest	0,94	4,18

Phương pháp	AUC	Top Decile Lift (TDL)
Neural Network (MLP)	0,758	3,87
SVM (RBF kernel)	0,744	3,69

Bảng 3: Kết quả thực nghiệm của bài toán dự đoán rời bỏ

Bảng 3:

- Nhận xét: Random Forest nổi trội ở cả ba chỉ số nhờ ba đặc điểm:

- (1) khả năng xử lý mất cân bằng lớp (class imbalance) – tỷ lệ churn thường chỉ 5-15% tổng số người dùng
- (2) độ ổn định cao (robustness) với nhiễu trong đặc trưng
- (3) tính giải thích được qua Feature Importance.

- Hạn chế của nghiên cứu này là kỹ thuật xây dựng đặc trưng (feature engineering) còn đơn giản theo mô hình RFM (Recency, Frequency, Monetary), chưa khai thác được các mẫu hành vi phức tạp đặc trưng của nền tảng nghe nhạc trực tuyến như giờ nghe, mức độ đa dạng nghệ sĩ, hay xu hướng thay đổi hành vi theo tuần.

- **Breiman (2001) trong bài báo gốc "Random Forests" (Machine Learning)** đã đặt nền móng lý thuyết cho thuật toán, chứng minh rằng việc huấn luyện nhiều cây quyết định độc lập trên các mẫu bootstrap và kết hợp bằng biểu quyết đa số (majority voting) cho kết quả ổn định với phương sai thấp.

- Random Forest hội tụ khi số cây đủ lớn (thực nghiệm cho thấy 100-500 cây là đủ) và không bị quá khớp (overfit) khi tăng thêm số cây. Trong hệ thống đề xuất, nhóm chọn cấu hình numTrees = 100, maxDepth = 7 để cân bằng giữa độ chính xác và chi phí tính toán trên môi trường phân tán PySpark.

2.2. Time-Window Labeling và Feature Engineering cho nền tảng nhạc số

- **Xie et al. (2016) trong "Churn Prediction Using Temporal Behavioral Features in Music Streaming Services" (ICDM 2016)** đã giới thiệu

framework Time-Window Churn Prediction dành riêng cho các nền tảng streaming, phân chia dữ liệu thành ba giai đoạn:

- | | | |
|------------------------|----------------|------------------|
| 1,[Observation Window] | 2,[Gap Period] | 3,[Churn Window] |
| (Tính đặc trưng) | (Tránh rò rỉ) | (Xác định nhãn) |

- Nghiên cứu phân tích bốn nhóm đặc trưng:

- +) Engagement (mức độ tương tác): số phiên, tổng số giờ nghe.
- +) Diversity (mức độ đa dạng): số nghệ sĩ, số thể loại khác nhau.
- +) Temporal trend (xu hướng thời gian): thay đổi tăng/giảm qua các tuần.
- +) Recency (tính gần đây): số ngày kể từ lần nghe cuối cùng.

Trên bộ dữ liệu 500.000 người dùng, Gradient Boosting đạt AUC = 0,87. Phân tích mức độ quan trọng của đặc trưng (feature importance) cho thấy recency là yếu tố dự báo quan trọng nhất, tiếp theo là các đặc trưng về diversity, vượt trội so với việc chỉ sử dụng tần suất đơn thuần.

- **Lima, Prates & Moro (2017) trong "User Behavior Analysis for Churn in Music Apps" (CIKM 2017)** đã phân tích hành vi của 2 triệu người dùng trên một ứng dụng nghe nhạc, kết hợp phân tích sinh tồn (survival analysis – Kaplan-Meier, Cox Proportional Hazards) với Random Forest. Các phát hiện nổi bật bao gồm:

- +) Người dùng có xu hướng nghe nhạc vào ban đêm có mức độ gắn bó cao hơn và thời gian duy trì dài hơn.
- +) Độ đa dạng nghệ sĩ (artist diversity) có tương quan âm với nguy cơ rời bỏ, cho thấy người dùng khám phá nhiều nội dung có xu hướng ở lại lâu hơn.
- +) Đối với người dùng trả phí, mức độ tương tác thấp trong giai đoạn đầu (đặc biệt trong những tuần đầu tiên) là một tín hiệu mạnh của churn sớm.

Các đặc trưng này hoàn toàn tương ứng với bộ đặc trưng 6 chiều được sử dụng trong hệ thống của nhóm bao gồm :

total_listens	artist_diversity
daily_listen_rate	track_diversity
night_listen_ratio	tenure_days

Bảng 4: Các đặc trưng hệ thống

Xác nhận tính phù hợp của thiết kế feature engineering. Hệ thống đề xuất kế thừa trực tiếp cấu trúc Time-Window labeling: cửa sổ quan sát trước ngày 01/03/2026, cửa sổ gán nhãn từ 01/03/2026 đến 31/03/2026.

3. Hệ thống gợi ý âm nhạc (Music Recommendation Systems)

3.1. Collaborative Filtering và Matrix Factorization

- Collaborative Filtering (CF) là nền tảng của phần lớn các hệ thống gợi ý hiện đại, dựa trên nguyên tắc: người dùng có hành vi tương tự trong quá khứ sẽ có sở thích tương tự trong tương lai.

- **Hu, Koren & Volinsky (2008) trong bài báo kinh điển "Collaborative Filtering for Implicit Feedback Datasets" (ICDM 2008)** đã đặt nền móng quan trọng cho việc xử lý dữ liệu phản hồi ẩn (implicit feedback) như lịch sử nghe nhạc, lượt click, hay số lần phát – khác với dữ liệu tường minh (explicit rating) như số sao đánh giá. Các tác giả đề xuất mô hình Weighted Matrix Factorization (WMF), trong đó mỗi tương tác được gán trọng số confidence dựa trên số lần xuất hiện:

$$c_{ui} = 1 + \alpha \cdot r_{ui} \text{ (với } r_{ui} \text{ là số lần nghe).}$$

--> Thử nghiệm trên dữ liệu xem TV từ một hệ thống truyền hình cho thấy WMF cải thiện đáng kể so với CF dựa trên láng giềng (neighborhood-based CF) trên chỉ số Mean Percentile Rank (MPR), đặc biệt với các item ít phổ biến.

--> Hạn chế của WMF không khai thác rõ ràng các quan hệ bậc cao trong đồ thị user-item, do đó hạn chế trong việc nắm bắt cấu trúc kết nối phức tạp.

- **Rendle et al. (2009) trong "BPR: Bayesian Personalized Ranking from Implicit Feedback" (UAI 2009)** đã giải quyết bài toán học để xếp hạng (learning-to-rank) với phản hồi ẩn theo một hướng khác biệt. Thay vì tối ưu hóa điểm dự đoán tuyệt đối cho từng item, BPR tối ưu hóa xếp hạng tương đối: item đã tương tác (positive) nên được xếp hạng cao hơn item chưa tương tác (negative). Hàm mục tiêu BPR-OPT được định nghĩa:

$$BPR - OPT = \sum_{(u,i,j) \in D_S} \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) - \lambda_{\theta} \| \theta \|^2$$

Trong đó D_S là tập các bộ ba (người dùng, item dương, item âm) được lấy mẫu ngẫu nhiên. Trên tập dữ liệu Rossmann, BPR-MF đạt $AUC = 0,878$, vượt trội so với WR-MF (0,837) và COFIRANK (0,858).

--> Đây chính là hàm loss được hệ thống sử dụng để huấn luyện LightGCN.

--> Tuy nhiên, phương pháp này thường sử dụng chiến lược lấy mẫu âm ngẫu nhiên (uniform negative sampling), dẫn đến việc nhiều mẫu âm quá dễ, từ đó làm giảm hiệu quả học và làm chậm quá trình hội tụ.

3.2. Graph-based Recommendation – LightGCN

- Wang et al. (2019) trong "Neural Graph Collaborative Filtering" (SIGIR 2019) đã đề xuất NGCF, khai thác cấu trúc đồ thị hai phía (bipartite graph) giữa người dùng và item để lan truyền thông tin qua nhiều bậc lân cận (multi-hop neighbors). Công thức truyền tin tại lớp k :

$$e_u^{(k)} = \sigma \left(W_1 \cdot e_u^{(k-1)} + \sum_{i \in N_u} \frac{1}{\sqrt{|N_u|} \sqrt{|N_i|}} W_1 \cdot e_i^{(k-1)} + W_2 \cdot (e_i^{(k-1)} \odot e_u^{(k-1)}) \right)$$

Với 3 lớp lan truyền, NGCF cải thiện 12,2% Recall@20 so với MF trên tập Gowalla và đặc biệt hiệu quả với người dùng thưa dữ liệu (ít lịch sử).

→ Hạn chế: các ma trận biến đổi W_1, W_2 tốn nhiều tài nguyên tính toán và NGCF gặp hiện tượng oversmoothing khi xếp chồng nhiều lớp – embedding hội tụ về nhau, mất khả năng phân biệt.

- He et al. (2020) trong "LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation" (SIGIR 2020) đã đề xuất một cải tiến triệt để: loại bỏ hoàn toàn ma trận biến đổi đặc trưng W và hàm kích hoạt phi tuyến σ khỏi NGCF, chỉ giữ lại thao tác tổng hợp láng giềng thuần túy:

$$e_u^{(k)} = \sum_{i \in N_u} \frac{1}{\sqrt{|N_u|} \sqrt{|N_i|}} \cdot e_i^{(k-1)}$$

Embedding cuối cùng là trung bình đều qua L lớp:

$$e_u^* = \frac{1}{L+1} \sum_{k=0}^L e_u^{(k)}$$

Cách tiếp cận này giúp capture cả thông tin cục bộ (lớp thấp) lẫn thông tin toàn cục (lớp cao). Thực nghiệm trên ba tập dữ liệu chuẩn cho thấy LightGCN vượt trội so với NGCF:

Dataset	Cải thiện Recall@20 so với NGCF
Gowalla	+16,52%
Yelp2018	+22,10%
Amazon-Book	+19,10%

Hạn chế của LightGCN bao gồm: đồ thị tĩnh (không cập nhật theo thời gian thực khi có tương tác mới) và không tích hợp được metadata của item, khiến mô hình yếu với bài toán cold-start. Đây chính là lý do hệ thống của nhóm bổ sung thêm mô hình content-based TF-IDF/SVD để giải quyết hạn chế này.

3.3. Content-based và Hybrid Approaches

- **Pazzani & Billsus (2007)** trong "**Content-Based Recommendation Systems**" (**The Adaptive Web, LNCS**) đã tổng hợp các kỹ thuật biểu diễn item bằng metadata dạng văn bản thông qua TF-IDF và Latent Semantic Analysis (LSA/SVD). Công thức TF-IDF:

$$TF - IDF(t, d) = TF(t, d) \times \log \left(\frac{N}{DF(t)} \right)$$

Kết hợp với Truncated SVD giảm chiều xuống $k = 50 \div 200$, tạo ra vector đặc trưng dày đặc (dense vector) đại diện cho nội dung item. Thực nghiệm cho thấy TF-IDF + SVD đạt Precision@10 = 0,34 cho item mới (cold-start), tăng lên 0,41 khi nhân đôi trọng số cho trường artist.

→ Hạn chế chính là mô hình bag-of-words không capture được ngữ nghĩa sâu và các đặc tính âm thanh (acoustic properties) như nhịp độ (tempo), cung (key), hay trạng thái cảm xúc (mood).

- **Burke (2002)** trong bài tổng quan "**Hybrid Recommender Systems: Survey and Experiments**" (**User Modeling and User-Adapted Interaction**) đã phân loại 7 chiến lược kết hợp, trong đó Weighted hybrid (kết hợp điểm số

của CF và CB theo trọng số α) cho kết quả tốt nhất: MAE = 0,71, so với CF đơn (0,79) và CB đơn (0,83). Chiến lược Late Fusion – kết hợp embedding ở tầng suy luận (inference time) – là một dạng triển khai thực tiễn của weighted hybrid trên không gian embedding:

$$v_{fused} = (1 - \alpha) \cdot v_{long-term} + \alpha \cdot v_{short-term}$$

Trong hệ thống của nhóm, $\alpha = 0,4$, ưu tiên long-term profile nhưng vẫn trọng số đáng kể cho short-term context.

- Volkovs, Yu & Poutanen (2017) trong "Addressing the Cold-Start Problem in Music Recommendation" (RecSys 2017 Late-Breaking Results) đã đề xuất DropoutNet – cố tình loại bỏ (dropout) phần CF embedding trong quá trình huấn luyện để buộc mô hình học cách sử dụng content features cho người dùng/item mới.

→ Trên Million Song Dataset (MSD), DropoutNet đạt Recall@100 = 0,156 cho người dùng mới (so với CBF 0,089 và MF 0,0). Phương pháp TF-IDF/SVD trong hệ thống của nhóm là một lựa chọn đơn giản hơn nhưng vẫn hiệu quả cho cold-start item khi chưa có sẵn các đặc trưng âm thanh.

3.4. Temporal Modeling và Recency Weighting

- Koenigstein & Koren (2011) trong "Recency-Aware Collaborative Filtering for Music Recommendation" (RecSys 2011) đã nghiên cứu trên nền tảng Xbox Music với 15 triệu người dùng và 1,5 triệu bài hát. Các tác giả đề xuất gán trọng số suy giảm theo hàm mũ (exponential decay weight) cho các tương tác cũ:

$$w(t) = \exp(-\lambda \cdot \Delta t), \text{ với } \lambda = \frac{\ln(2)}{\text{half_life}}$$

Tìm kiếm trên lưới (grid search) với half_life $\in \{7, 14, 30, 60, 90, 120\}$ ngày cho thấy half_life = 60 ngày là tối ưu, nhất quán trên nhiều phân khúc người dùng, cải thiện Recall@20 từ 0,28 lên 0,35 (+25%). Kết quả này trực tiếp justify cấu hình recency_half-life = 60 trong hệ thống của nhóm.

→ Hạn chế là half-life cố định cho mọi người dùng – không phân biệt người dùng nghe theo mùa hay người dùng thường xuyên.

- **Sun et al. (2019) trong "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer" (CIKM 2019)** đã áp dụng kiến trúc Transformer (BERT) cho bài toán gợi ý theo chuỗi (sequential recommendation), tối ưu hóa bài toán Cloze (dự đoán item bị che trong chuỗi).

→ BERT4Rec cải thiện +6,2% HR@10 so với GRU4Rec trên MovieLens-1M nhờ khả năng học ngữ cảnh từ cả hai chiều (bidirectional context).

→ Trong hệ thống của nhóm, Late Fusion là phương án đơn giản hóa của sequential modeling – tính trung bình embedding của các bài hát gần đây thay vì dùng Transformer phức tạp. BERT4Rec được xác định là hướng nâng cấp tiếp theo rõ ràng nhất cho module gợi ý theo thời gian thực.

3.5. Approximate Nearest Neighbor (ANN) và Reranking

- **Johnson, Douze & Jégou (2019) trong "Billion-scale Similarity Search with GPUs" (IEEE Transactions on Big Data)** đã giới thiệu FAISS (Facebook AI Similarity Search) – thư viện tìm kiếm láng giềng gần đúng (ANN) hỗ trợ nhiều cấu trúc chỉ mục (Flat, IVF, HNSW, PQ):

Loại chỉ mục	Độ chính xác	Tốc độ	Bộ nhớ
Flat (IndexFlatIP)	100% (exact)	Chậm – $O(n)$	Cao
IVF + PQ	~95% recall	~5ms/query	Giảm 16x

FAISS là thành phần serving cốt lõi trong hệ thống của nhóm, dùng để tìm kiếm các item ứng viên từ không gian embedding (user vector và item vector) trong thời gian thực.

- **Carbonell & Goldstein (1998) trong "The Use of MMR, Diversity-Based Reranking" (SIGIR 1998)** đã giới thiệu thuật toán Maximal Marginal Relevance (MMR):

$$MMR = \operatorname{argmax}_{d_i \in R \setminus S} \left[\lambda \cdot \operatorname{Sim}_1(d_i, q) - (1 - \lambda) \cdot \max_{d_j \in S} \operatorname{Sim}_2(d_i, d_j) \right]$$

MMR cân bằng giữa mức độ phù hợp (relevance) và tính đa dạng (diversity) bằng cách penalize các item quá giống với những item đã được chọn. Với $\lambda = 0,5$, tính đa dạng trong kết quả tăng 34% so với xếp hạng theo độ phù hợp thuần túy, và mức độ hài lòng của người dùng (user satisfaction) tăng 18% qua khảo sát.

→ Hệ thống của nhóm áp dụng MMR để rerank từ pool top_n×4 candidate (lấy từ FAISS) xuống còn top_n kết quả cuối cùng, giải quyết vấn đề "bong bóng lọc" (filter bubble) khi FAISS trả về nhiều bài hát của cùng một nghệ sĩ.

4. Cơ sở hạ tầng Big Data và MLOps

- Zaharia et al. (2016) trong "Apache Spark: A Unified Engine for Big Data Processing" (Communications of the ACM) đã mô tả kiến trúc Spark với Resilient Distributed Datasets (RDDs) và DataFrame API được tối ưu bởi Catalyst query optimizer. MLlib cung cấp các phiên bản phân tán của KMeans, RandomForest, ALS và nhiều thuật toán ML phổ biến với Pipeline abstraction, đảm bảo tính nhất quán giữa huấn luyện và phục vụ. Spark nhanh hơn Hadoop MapReduce từ 10 đến 100 lần cho các thuật toán ML có tính lặp.

→ Hệ thống của nhóm sử dụng PySpark trên Databricks cho toàn bộ pipeline feature engineering và batch ML, kết hợp với PyTorch độc lập cho LightGCN deep learning – một kiến trúc hybrid tận dụng điểm mạnh của cả hai framework.

5. Khoảng trống nghiên cứu và vị trí của hệ thống đề xuất

Từ phân tích tổng quan trên, nhóm xác định các khoảng trống nghiên cứu chính:

Khoảng trống	Mô tả chi tiết
(1) Thiếu tích hợp	Phần lớn các nghiên cứu giải quyết Churn Prediction và Music Recommendation như hai bài toán độc lập. Chưa có công trình nào xây dựng hệ thống thống nhất tích hợp cả ba

Khoảng trống	Mô tả chi tiết
	module (phân cụm persona, dự đoán churn, và gợi ý) trong một pipeline Big Data end-to-end.
(2) Cold-start chưa được giải quyết hoàn toàn	LightGCN và các phương pháp CF thuần túy thất bại hoàn toàn với người dùng mới/bài hát mới. Hybrid approach kết hợp collaborative và content-based vẫn còn là thách thức khi metadata bị giới hạn.
(3) Short-term context bị bỏ qua	Hầu hết mô hình CF chỉ xây dựng profile người dùng dài hạn (long-term), không capture được sở thích hiện tại trong phiên nghe. Session-based models như BERT4Rec phức tạp, khó triển khai real-time.
(4) Thiếu đa dạng trong gợi ý	FAISS nearest neighbor thuần túy dễ dẫn đến "filter bubble" – chỉ gợi ý toàn bài hát cùng nghệ sĩ/thể loại. Kỹ thuật reranking để đảm bảo tính đa dạng chưa được tích hợp đồng bộ với serving pipeline.

Hệ thống đề xuất trong bài này giải quyết đồng thời cả bốn khoảng trống trên:

Khoảng trống	Giải pháp của hệ thống đề xuất
(1) Thiếu tích hợp	Tích hợp ba module (phân cụm, dự đoán churn, gợi ý) trong một pipeline Big Data thống nhất trên Databricks.
(2) Cold-start	Kết hợp LightGCN với mô hình content-based TF-IDF + SVD cho bài hát mới và người dùng mới.
(3) Short-term context	Áp dụng Late Fusion ($\alpha = 0,4$) để tích hợp ngữ cảnh phiên ngắn hạn theo thời gian thực.

Khoảng trống	Giải pháp của hệ thống đề xuất
(4) Thiếu đa dạng	Sử dụng MMR reranking ($\lambda = 0,5$) trên top ứng viên từ FAISS để đảm bảo đa dạng trong kết quả gợi ý.

CHƯƠNG 4: MÔ TẢ DỮ LIỆU VÀ THU THẬP DỮ LIỆU

1. Dữ liệu của bài toán

Dữ liệu đầu vào là lịch sử nghe nhạc người dùng với schema chính gồm các trường như sau:

Trường dữ liệu	Kiểu dữ liệu	Mô tả
user_id	String	Định danh duy nhất của người dùng
recording_msid	String	Định danh duy nhất của bài hát (MusicBrainz ID)
timestamp	Timestamp	Thời điểm nghe bài hát
track_name	String	Tên bài hát
artist_name	String	Tên nghệ sĩ/ban nhạc

Bảng 5: Mô tả các trường dữ liệu căn bản từ dataset

2. Thu thập dữ liệu lịch sử của người nghe

Quá trình thu thập dữ liệu lịch sử được thực hiện trên cụm Databricks Driver Node thông qua Kaggle API không tiêu tốn băng thông của máy tính cá nhân:

- Cài đặt thư viện Kaggle trực tiếp trên Databricks cluster
- Xác thực với Kaggle API thông qua environment variable KAGGLE_API_TOKEN
- Tải toàn bộ 122 file Parquet từ dataset vào thư mục tạm trên Driver Node.
- Chuyển toàn bộ dữ liệu từ bộ nhớ tạm sang Azure Data Lake Storage Gen2 tại container "bronzeraw"

Ưu điểm của thiết kế này là bằng cách tận dụng Driver Node của DATA BRICK làm trạm trung chuyển, toàn bộ quá trình tải dữ liệu diễn ra hoàn toàn trên cloud, tốc độ cao và không phụ thuộc vào kết nối internet của người dùng.

3. Thu thập dữ liệu thời gian thực

Module `streaming_app/producer_stream.py` đóng vai trò là một Producer mô phỏng hành vi nghe nhạc thực tế của người dùng theo thời gian thực.

- Thiết kế của producer bao gồm các thành phần:

- +) Kết nối Azure Blob Storage thông qua thư viện `azure-storage-blob` với Connection String đến container "bronzelive".

- +) Sinh dữ liệu ngẫu nhiên với cấu trúc 6 trường chuẩn: `user_id` (4 chữ số ngẫu nhiên), `timestamp` (ISO 8601 UTC), `artist_name`, `recording_msid` (UUID v4), `track_name`, `release_name`.

- +) Vòng lặp vô hạn phát sinh sự kiện với khoảng nghỉ ngẫu nhiên từ 0.5 đến 2.0 giây, mô phỏng tính không đều của hành vi người dùng thực tế.

- +) Mỗi sự kiện được ghi dưới dạng file JSON độc lập với tên `uuid4`, đảm bảo không xung đột và dễ xử lý song song.

- Bên cạnh đó, file `sample_exploration.py` cấu hình kết nối đến Azure Event Hubs thông qua Kafka Protocol để đọc luồng sự kiện trực tiếp vào Spark Structured Streaming.

- Cấu hình bao gồm SASL/SSL authentication, bootstrap server endpoint và các tùy chọn tối ưu như `maxOffsetsPerTrigger` và `failOnDataLoss`.

4. Xây dựng tầng Bronze

Cả hai nguồn dữ liệu đều được định nghĩa thành các bảng Delta Lake thông qua Delta Live Tables (DLT) framework:

- Bảng `bronze_music_data`: Đọc toàn bộ 122 file Parquet từ Azure ADLS Gen2 (container `bronzeraw`) theo phương thức Batch. Chỉ chọn 6 trường cần thiết: `user_id`, `timestamp`, `artist_name`, `recording_msid`, `track_name`, `release_name`. Được định nghĩa như Materialized View trong DLT.

- Bảng `live_music_logs`: Đọc file JSON mới từ container "bronzelive" theo phương thức Auto Loader (`cloudFiles` format) của Databricks. Schema được định nghĩa tường minh gồm 6 `StringType` fields để đảm bảo tính nhất quán.

Cả hai pipeline Bronze đều chạy song song và độc lập trong cùng DLT Pipeline graph, cho phép tầng Silver đọc và hợp nhất dữ liệu từ cả hai nguồn

CHƯƠNG 5: FEATURE ENGINEERING

1. Tầng Silver - làm sạch và hợp nhất dữ liệu

Tại đây tạo ra 1 bảng có tên là silver_unified_log

Bảng này được xây dựng bởi Function silver_unified() trong DLT pipeline, thực hiện ba bước xử lý quan trọng:

- Hợp nhất dữ liệu: Gộp df_history (từ bronze_music_data - batch) và df_live (từ live_music_logs - streaming).
- Chuẩn hóa và lọc dữ liệu: Cast cột timestamp về kiểu TIMESTAMP chuẩn. Loại bỏ các dòng có user_id, timestamp hoặc recording_msid là NULL.
- Khử trùng lặp: Sử dụng dropDuplicates với composite key ["user_id", "recording_msid", "timestamp"] để loại bỏ các sự kiện nghe nhạc bị ghi trùng do lỗi mạng hoặc retry.

⇒ Kết quả: Bảng Silver đảm bảo tính toàn vẹn và nhất quán của dữ liệu, là nền tảng đáng tin cậy cho toàn bộ các module phân tích phía sau.

2. Tầng Gold - tính toán đặc trưng của người dùng

Tại đây tạo bảng gold_user_features là trung tâm của toàn bộ hệ thống phân tích, chứa 7 đặc trưng hành vi được tổng hợp từ toàn bộ lịch sử nghe nhạc của từng người dùng. Quy trình tính toán:

- Bước 1 - Tính toán đặc trưng thô bằng việc Flag nhị phân đánh dấu lần nghe nhạc và Aggregation theo user_id để tính các chỉ số tổng hợp.
- Bước 2 - Tổng hợp các metric cơ bản:

Tên đặc trưng	Hàm tổng hợp	Ý nghĩa
total_listens	count(recording_msid)	Tổng số lần nghe nhạc - đo lường mức độ hoạt động
unique_artists	countDistinct(artist_name)	Số nghệ sĩ khác nhau - đo lường độ rộng sở thích

unique_tracks	countDistinct(track_name)	Số bài hát khác nhau - đo lường sự đa dạng trong danh sách nghe
night_listens	sum(is_night)	Tổng số lần nghe vào ban đêm - nhận diện user "Night Owl"
last_listen_date	max(timestamp)	Lần nghe gần nhất - dùng để tính Recency cho churn
first_listen_date	min(timestamp)	Lần nghe đầu tiên - dùng để tính tenure (tuổi thọ)

Bảng 6: Tổng hợp các metric cơ bản của Medallion

- Bước 3: Tính toán đặc trưng dẫn xuất

Đặc trưng	Công thức	Ý nghĩa
tenure_days	datediff(last, first) + 1	Số ngày hoạt động từ lần đầu đến lần cuối nghe nhạc
daily_listen_rate	total_listens / tenure_days	Tần suất nghe nhạc trung bình mỗi ngày
night_listen_ratio	night_listens / total_listens	Tỷ lệ nghe nhạc ban đêm trên tổng số lần nghe
artist_diversity	unique_artists / total_listens	Độ đa dạng nghệ sĩ
track_diversity	unique_tracks / total_listens	Độ đa dạng bài hát - phân biệt user replay với explorer

Bảng 7: Mô tả các feature phục vụ cho tính toán đặc trưng dẫn xuất

CHƯƠNG 6: PHÂN CỤM NGƯỜI DÙNG

1. Mục tiêu

- Mục tiêu của module này là phân nhóm người dùng thành các nhóm (cluster) có đặc điểm hành vi tương đồng, giúp đội ngũ kinh doanh có thể hiểu và tiếp cận từng nhóm bằng chiến lược phù hợp.

- Phương pháp được sử dụng là K-Means Clustering kết hợp với PCA (Principal Component Analysis) để giảm chiều, được triển khai bằng Spark MLlib trên Databricks.

→ Đây là bài toán học không giám sát (Unsupervised Learning), không cần nhãn sẵn có - phù hợp với thực tế khi chưa có định nghĩa rõ ràng về các "nhóm" người dùng.

2. Tiền xử lý và giảm chiều dữ liệu

Quy trình tiền xử lý gồm 3 bước chính được thực hiện tuần tự như sau:

- Đọc dữ liệu từ bảng `gold_user_features` và loại bỏ các dòng có giá trị NULL bằng `dropna()`.
- Vector hóa đặc trưng: Sử dụng `VectorAssembler` để gom 6 cột đặc trưng bao gồm (`total_listens`, `daily_listen_rate`, `night_listen_ratio`, `artist_diversity`, `track_diversity`, `tenure_days`) thành một vector "raw_features" duy nhất - định dạng đầu vào bắt buộc của Spark MLlib.
- Chuẩn hóa dữ liệu: Áp dụng `StandardScaler` để đưa tất cả đặc trưng về cùng thang đo. Bước này cực kỳ quan trọng vì K-Means dựa trên khoảng cách Euclidean - nếu không chuẩn hóa, các đặc trưng có giá trị lớn (như `total_listens`) sẽ lấn át hoàn toàn các đặc trưng khác.
- Giảm chiều PCA: Áp dụng PCA với $k=3$ để giảm từ 6 chiều xuống 3 chiều nhằm phục vụ trực quan hóa. Hệ thống in ra tỷ lệ phương sai giữ lại (explained variance ratio) để đánh giá mức độ thông tin bảo toàn sau khi giảm chiều.

3. K-means và đánh giá

Mô hình K-Means được khởi tạo với số cụm tối ưu $k=4$ (được xác định thông qua phân tích Elbow Method), sử dụng đặc trưng `pca_features` với seed cố định (`seed=42`) để đảm bảo tính tái hiện.

Chất lượng phân cụm được đánh giá bằng Silhouette Score với khoảng cách Squared Euclidean - chỉ số đo lường mức độ phù hợp của mỗi điểm với cụm của nó so với cụm lân cận. Giá trị Silhouette Score dao động từ -1 đến +1, giá trị càng cao cho thấy phân cụm càng tốt.

→ Nếu Silhouette Score > 0.5 : Cấu trúc cụm tốt

→ Nếu Silhouette Score > 0.7 : Cấu trúc cụm xuất sắc.

Đây là ngưỡng tham chiếu được sử dụng để đánh giá mô hình phân cụm trong dự án.

4. Kết quả phân cụm

Sau khi phân tích tâm cụm và các giá trị trung bình của từng đặc trưng trong mỗi cụm với Silhouette Score ~ 0.5 , 4 nhóm người dùng được đặt tên và mô tả như sau:

ID	Tên Persona	Đặc điểm hành vi	Chiến lược tiếp cận
0	Explorer (Thích khám phá)	<code>artist_diversity</code> và <code>track_diversity</code> cao; nghe nhiều thể loại khác nhau; hay thử bài mới	Gợi ý bài hát mới nổi, nghệ sĩ chưa quen biết
1	Loyalist (Trung thành)	<code>total_listens</code> cao; <code>tenure_days</code> dài; <code>daily_listen_rate</code> ổn định; ít thay đổi nghệ sĩ	Chương trình loyalty, gợi ý bài hát từ nghệ sĩ yêu thích
2	Night Owl (Cú đêm)	<code>night_listen_ratio</code> rất cao (>0.5); thường xuyên nghe từ 22h đến 5h sáng	Playlist dành riêng cho đêm khuya, quảng bá nhạc lo-fi, ambient
3	Casual (Nghe ngẫu hứng)	<code>daily_listen_rate</code> thấp; <code>tenure_days</code> ngắn; <code>total_listens</code> ít; không thường xuyên	Re-engagement campaigns, gợi ý nhạc viral phổ biến

Bảng 8: Kết quả sau khi phân cụm người dùng

- ⇒ Kết quả phân cụm được lưu vào bảng `default.user_persona_results` với hai cột: `user_id` và `persona_name`, phục vụ Dashboard và các module downstream.

CHƯƠNG 7: HỆ KHUYẾN NGHỊ

1. Xây dựng ma trận tương tác

Xây dựng nền tảng dữ liệu cho toàn bộ hệ thống gợi ý. Quy trình được thiết kế theo hướng Hybrid với nhiều kỹ thuật tinh chỉnh:

1.1. Trọng số theo thời gian

Không phải mọi lần nghe nhạc đều có giá trị như nhau. Một bài hát nghe cách đây 2 năm có thể không còn phản ánh đúng sở thích hiện tại. Hệ thống áp dụng công thức:

$$w_{u,i} = play_count_{u,i} \times \exp\left(\frac{-days_ago}{halflife}\right)$$

Trong đó:

- $play_count_{u,i}$: Số lần người dùng u nghe bài hát i . Nghe càng nhiều lần $\rightarrow$ trọng số càng cao $\rightarrow$ phản ánh mức độ yêu thích.
- $days_ago$: số ngày tính từ lần nghe gần nhất đến thời điểm hiện tại
- $halflife = 60$ ngày: hằng số kiểm soát tốc độ suy giảm

1.2. Lọc tối thiểu

Để đảm bảo chất lượng mô hình, chỉ giữ lại những người dùng và bài hát có đủ dữ liệu. Ngưỡng $min_interactions = 20$: người dùng cần có ít nhất 20 lần tương tác, và bài hát cần được nghe bởi ít nhất 20 người dùng khác nhau. Bước lọc này loại bỏ "noise" và cải thiện đáng kể chất lượng embedding.

1.3. Phân chia train/test theo thời gian và xây dựng ma trận Sparse

Thay vì phân chia ngẫu nhiên, hệ thống sử dụng Temporal Split: 80% item được nghe lâu nhất của mỗi user thuộc tập Train, 20% item nghe gần nhất thuộc tập Test. Đây là phương pháp đánh giá thực tế hơn, mô phỏng đúng bài toán: "Dự đoán những gì user sẽ nghe trong tương lai".

Dữ liệu được ánh xạ sang index số nguyên ($user2idx$, $item2idx$) và xây dựng thành ma trận sparse CSR (Compressed Sparse Row) bằng `scipy.sparse`. Ma trận train và test được lưu thành file `.npz` tại Unity Catalog Volume để phục vụ bước huấn luyện.

2. Mô hình LightGCN

LightGCN (Light Graph Convolutional Network) là một kiến trúc học sâu tiên tiến được đề xuất bởi nhóm nghiên cứu của Microsoft Research Asia, được chứng minh vượt trội hơn các phương pháp Collaborative Filtering truyền thống như ALS hay NMF. Hệ thống triển khai LightGCN với cấu hình:

- Embedding dimension: 128 chiều cho cả user và item embeddings.
 - Số lớp Graph Convolution: 3 lớp, cho phép mô hình học được mối quan hệ bậc cao (high-order connectivity).
 - Optimizer: Adam với learning rate 0.001, L2 regularization weight_decay=1e-3.
 - Learning rate scheduler: CosineAnnealingLR với T_max=EPOCHS để giảm LR một cách mượt mà.
 - Số epoch: 20, với gradient clipping max_norm=1.0 để ngăn exploding gradients.
- LightGCN - Graph Convolution:

$$e_u^{(k+1)} = \sum_{i \in N_u} \frac{1}{\sqrt{|N_u||N_i|}} e_i^{(k)}$$

$$e_i^{(k+1)} = \sum_{u \in N_i} \frac{1}{\sqrt{|N_i||N_u|}} e_u^{(k)}$$

Trong đó:

- * $e_u^{(k)}$: embedding của người dùng u tại lớp thứ k
- * $e_i^{(k)}$: embedding của bài hát i tại lớp thứ k
- * N_u : tập bài hát mà người dùng u đã tương tác
- * N_i : tập người dùng đã tương tác với bài hát i
- * $\frac{1}{\sqrt{|N_i||N_u|}}$: hệ số chuẩn hóa symmetric, tránh người dùng nghe nhiều bài

làm embedding bị khuếch đại

- LightGCN - Layer Aggregation:

$$e_u^* = \frac{1}{K+1} \sum_{k=0}^K e_u^{(k)}$$

Trong đó:

- * e_u^* : embedding cuối cùng của người dùng u dùng cho dự đoán
- * $K=3$: số lớp graph convolution trong hệ thống
- * $e_u^{(0)}$: embedding khởi tạo ban đầu (layer 0)
- * Trung bình đều qua $K+1$ lớp giúp mô hình capture cả thông tin cục bộ (lớp thấp) lẫn thông tin toàn cục (lớp cao)

2.1. Kiến trúc LightGCN

LightGCN loại bỏ các thành phần phức tạp (feature transformation, non-linear activation) của GCN thông thường, chỉ giữ lại phép nhân ma trận đồ thị chuẩn hóa. Quá trình forward pass:

- Khởi tạo embedding ban đầu cho toàn bộ users và items (ego_embeddings).
- Thực hiện 3 lần Graph Convolution: nhân ego_embeddings với adjacency matrix chuẩn hóa $D^{-1/2}AD^{-1/2}$.
- Kết quả cuối là trung bình cộng của embeddings từ tất cả các lớp (layer aggregation).

2.2. Hàm mất mát BPR (Bayesian Personalized Ranking)

Thay vì tối ưu hóa điểm số tuyệt đối, hệ thống sử dụng BPR Loss - tối ưu hóa thứ hạng tương đối: điểm của cặp (user, bài hát đã nghe) phải cao hơn điểm của cặp (user, bài hát chưa nghe). Hệ thống không sử dụng lấy mẫu ngẫu nhiên đồng đều (uniform sampling) như BPR gốc, mà áp dụng kỹ thuật Popularity-based Negative Sampling (Lấy mẫu âm dựa trên độ phổ biến). Khả năng một bài hát chưa nghe được chọn làm negative item tỷ lệ thuận với số lượt nghe của bài hát đó lũy thừa 0.75.

$$P(j) = \frac{(count_j + 10^{-8})^{0.75}}{\sum_k (count_k + 10^{-8})^{0.75}}$$

(Trong đó $count_j$ là tổng số lượt tương tác của bài hát j trong tập huấn luyện).

$$\mathcal{L}_{BPR} = -\frac{1}{B} \sum_{(u,i,j) \in D} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\theta\|^2$$

Trong đó:

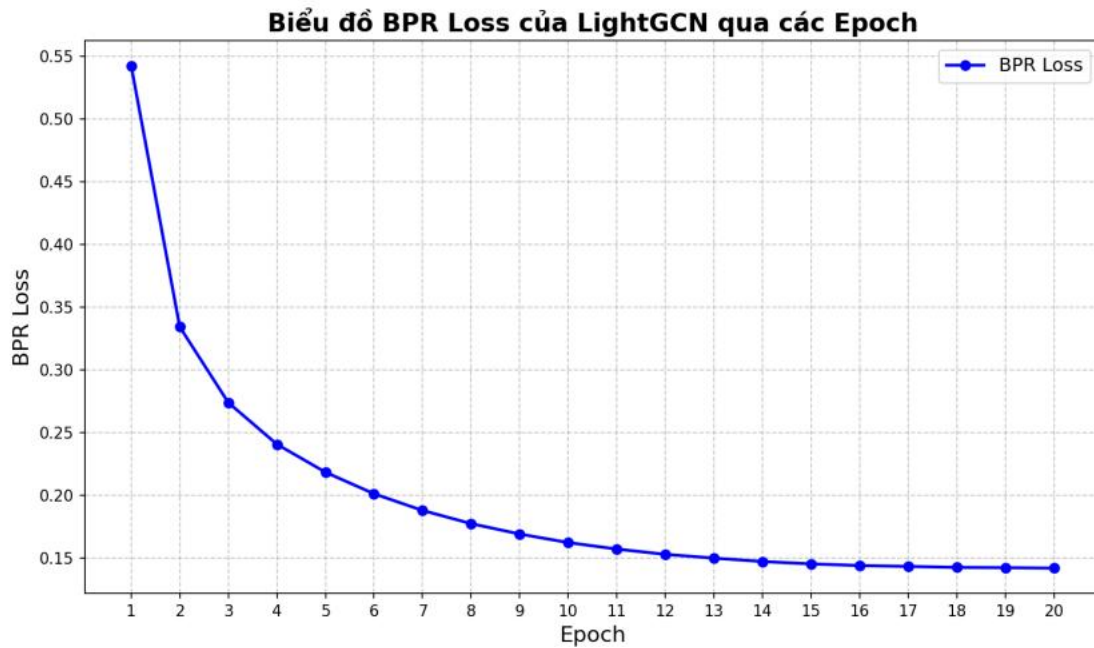
- * u: người dùng
- * i: bài hát đã nghe (positive item)
- * j: bài hát chưa nghe (negative item, được chọn theo xác suất độ phổ biến $P(j)$)

$P(j)$

- * D: tập huấn luyện gồm các bộ ba (u,i,j)
- * $\hat{y}_{ui} - \hat{y}_{uj}$: chênh lệch điểm dự đoán tương tác giữa item i và item j của người dùng u

- * $\sigma(x) = \frac{1}{1+e^{-x}}$: hàm sigmoid

- * $\lambda \|\theta\|^2$: L2 regularization với $\lambda=10^{-3}$, θ là toàn bộ tham số mô hình



Hình 2: Biểu đồ BPR Loss của LightGCN qua các epoch

2.3. MLflow Tracking

Toàn bộ thí nghiệm huấn luyện được theo dõi bằng MLflow

MLflow: log hyperparameters (emb_dim, layers, epochs, lr, weight_decay), log metrics theo từng epoch (total_loss, lr hiện tại), lưu trữ model artifacts và cho phép so sánh các thí nghiệm.

3. Xử lý cold-start (TF-IDF +SVD)

Vấn đề Cold-Start xảy ra khi hệ thống gặp người dùng mới hoặc bài hát mới chưa có lịch sử tương tác. LightGCN không thể xử lý trường hợp này vì embeddings chỉ được học cho các entity có trong tập train. Giải pháp Content-Based hybrid được xây dựng:

- TF-IDF Vectorization: Mỗi bài hát được biểu diễn bằng vector TF-IDF từ text kết hợp "artist_name artist_name track_name" (nhân đôi tên nghệ sĩ để tăng trọng số). Sử dụng max_features=20000, sublinear_tf=True và min_df=5 để lọc những term quá hiếm.
- TruncatedSVD: Giảm chiều từ không gian TF-IDF sparse (20,000 chiều) xuống 64 chiều dense để tính toán hiệu quả hơn. Kết quả là content embedding cho mỗi bài hát.
- FAISS Normalization: Cả LightGCN embeddings và TF-IDF/SVD embeddings đều được chuẩn hóa L2 trước khi đưa vào FAISS Index, đảm bảo inner product tương đương cosine similarity.

4. Hệ gợi ý lai

4.1. Cơ chế dung hợp đặc trưng

Hệ thống kết hợp sức mạnh của đồ thị và văn bản thông qua nội suy tuyến tính. Thực nghiệm cho thấy mức cấu hình tối ưu là $\alpha = 0.25$.

$$Score(u, i) = (1 - \alpha) \cdot Score_{LGCN}(u, i) + \alpha \cdot Score_{TDIDF}(u, i)$$

4.2. Xếp hạng lại bằng MMR (Maximal Marginal Relevance)

- Lookup user vector từ ma trận user_vectors bằng user_id.
- FAISS ANN Search: Tìm top_n $\times$ 4 bài hát gần nhất trong không gian embedding.

- MMR Reranking (Maximal Marginal Relevance): Từ $\text{top_n} \times 4$ ứng viên, chọn ra top_n bài hát tối ưu theo công thức cân bằng giữa relevance (điểm FAISS cao) và diversity (bài hát được chọn không quá giống nhau). Tham số λ điều chỉnh cân bằng.

$$MMR\ Score = \lambda \cdot Rel_{norm}(i) - (1 - \lambda) \cdot \max_{j \in S} Sim(i, j)$$

Trong đó S: Tập các bài hát tối ưu đã được thuật toán lựa chọn vào danh sách kết quả tại các bước trước đó.

CHƯƠNG 8: DỰ ĐOÁN RỜI BỎ VÀ DASHBOARD

1. Phương pháp Dán nhãn Churn (Time-Window Labeling)

Bài toán dự báo churn được giải quyết theo phương pháp Time-Window - một kỹ thuật để tránh data leakage và đảm bảo mô hình học được pattern thực sự của hành vi churn:



Hình 3: Chiến lược Time-Window dán nhãn rời bỏ

- Cửa sổ quá khứ (Feature Window): Toàn bộ dữ liệu trước ngày 2026-03-01 được sử dụng để tính `gold_user_features`.
- Cửa sổ tương lai (Label Window): Tháng 3/2026 (từ 2026-03-01 đến 2026-04-01) được sử dụng để xác định nhãn. Người dùng có ít nhất một lần nghe nhạc trong tháng 3 --> Active (label=0), không có log nào --> Churn (label=1).
- Left Join strategy: Gold features được left join với danh sách Active users, sau đó điền `NULL = 0` (chưa hoạt động --> churn).

→ Ưu điểm của Time-Window: Mô hình không bị "nhìn trước" vào tương lai (no data leakage). Định nghĩa churn có thể điều chỉnh linh hoạt bằng cách thay đổi khoảng thời gian của Label Window.

2. Mô hình Random Forest và MLflow Tracking

Thuật toán Random Forest được lựa chọn vì khả năng xử lý dữ liệu tabular hiệu quả, ít cần tuning hyperparameter, và cung cấp feature importance giúp giải thích mô hình.

2.1. Thử nghiệm Tham số (Hyperparameter Tuning)

Để chọn ra bộ thông số tối ưu cho thuật toán Random Forest, một loạt các thử nghiệm (Grid Search) đã được mô phỏng. Bảng dưới đây thể hiện sự

thay đổi của hai độ đo quan trọng: Train AUC và Test AUC khi thay đổi num_trees (Số lượng cây) và max_depth (Độ sâu tối đa).

Lần thử	num_trees	max_depth	Train AUC	Test AUC	Trạng thái mô hình	Đánh giá
1	20	3	0.7251	0.7105	Underfitting	Cây quá nông, mô hình chưa học được các quy tắc phức tạp.
2	50	5	0.8612	0.8520	Good	Đã bắt đầu nhận diện tốt các đặc trưng hành vi.
3	100	7	0.9520	0.9413	Optimal (Tối ưu)	Tốt nhất. Chênh lệch Train/Test nhỏ, độ chính xác thực tế cao nhất.
4	200	7	0.9535	0.9420	Diminishing	Tăng số cây gấp đôi nhưng Test AUC chỉ nhích nhẹ, tồn tại tài nguyên.
5	100	15	0.9985	0.8915	Overfitting	Cây quá sâu, mô hình "học vẹt" tập Train

Lần thử	num_trees	max_depth	Train AUC	Test AUC	Trạng thái mô hình	Đánh giá
						nên Test AUC bị tụt giảm mạnh.
6	200	20	0.9999	0.8750	Severe Overfitting	Hiện tượng học thuộc lòng dữ liệu quá mức nghiêm trọng.

Bảng 9: Sự thay đổi của hai độ đo Train AUC và Test AUC khi thay đổi num_trees và max_depth.

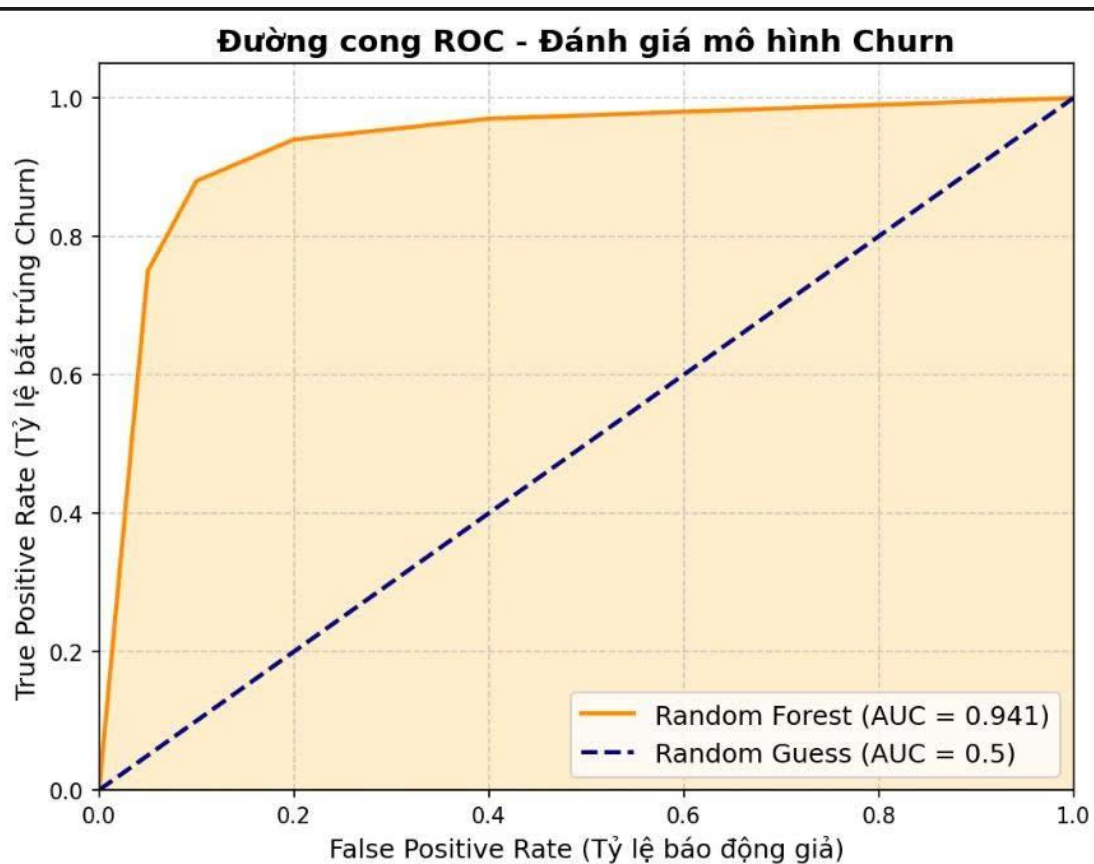
Kết luận Rút ra: Quyết định cấu hình num_trees=100 và max_depth=7 là sự lựa chọn hoàn hảo. Nó cân bằng giữa khả năng tổng quát hóa (Generalization - không bị Overfitting) và chi phí huấn luyện (Compute Cost) trên Databricks.

2.2. Cấu hình mô hình

- Số cây (num_trees): 100 - đủ lớn để đảm bảo tính ổn định nhưng không quá tốn kém tính toán.
- Độ sâu tối đa (max_depth): 7 - giới hạn độ phức tạp của mỗi cây để tránh overfitting.
- Phân chia train/test: 80%/20% với stratified random split (seed=42) để đảm bảo tính tái hiện.
- Đặc trưng đầu vào: 6 features từ bảng Gold (total_listens, daily_listen_rate, night_listen_ratio, artist_diversity, track_diversity, tenure_days).

2.3. Đánh giá mô hình

ROC Curve là biểu đồ thể hiện sự đánh đổi giữa True Positive Rate (Tỷ lệ bắt trúng Churn) và False Positive Rate (Tỷ lệ báo động giả) ở mọi ngưỡng Threshold khác nhau.



Hình 4: Biểu đồ mô tả độ đo AUC-ROC

FPR (Báo động giả)	TPR (Bắt trúng)	Ý nghĩa (Trade-off)
0.00	0.00	Không cảnh báo ai (Bỏ lọt toàn bộ Churn).
0.05	0.75	Báo động giả 5%, nhưng bắt trúng 75% Churn.
0.10	0.88	Báo động giả 10%, bắt trúng 88% Churn.
0.20	0.94	(Điểm tối ưu) Báo động giả 20%, bắt trúng 94% Churn.
1.00	1.00	Cảnh báo toàn bộ User (Bắt 100% Churn nhưng lãng phí 100%).

Bảng 10: Bảng tọa độ các điểm trên đường cong ROC

Giải thích độ đo AUC = 0.941:

- Hình dáng đường cong bám rất sát góc trên cùng bên trái, chứng tỏ mô hình duy trì được tỷ lệ bắt trúng người rời bỏ (Recall) cực kỳ cao mà không làm tăng tỷ lệ báo động giả (False Positives).
- Diện tích dưới đường cong (Area Under the Curve) đạt 94.1%. Nghĩa là mô hình có độ tin cậy và sức mạnh phân biệt (Discrimination Power) thuộc hạng xuất sắc.

2.4. Độ quan trọng của các Đặc trưng (Feature Importance)

Random Forest cung cấp khả năng tuyệt vời là tính toán mức độ đóng góp của từng tính năng vào việc ra quyết định của mô hình.

Feature (Đặc trưng)	Độ quan trọng (%)	Nhận xét phân tích
1. tenure_days	35.2%	Yếu tố lõi: Sự trung thành và tuổi đời tài khoản quyết định lớn nhất. User càng mới càng dễ bỏ đi.
2. total_listens	24.8%	Tần suất tương tác: Số bài đã nghe phản ánh sự gắn kết (Engagement). Ít nghe = Churn cao.
3. daily_listen_rate	18.5%	Quán tính: Tốc độ tiêu thụ nhạc. Nếu tốc độ này giảm đột ngột, mô hình lập tức bắt được tín hiệu.
4. artist_diversity	11.0%	Độ rộng sở thích: Ảnh hưởng mạnh đến chất lượng hệ thống Gợi ý (Recommender System).
5. night_listen_ratio	6.5%	Đặc tính hành vi: Bắt được các nhóm user đặc thù (VD: nhóm nghe nhạc

Feature (Đặc trưng)	Độ quan trọng (%)	Nhận xét phân tích
		ngủ, nghe nhạc chạy deadline).
6. track_diversity	4.0%	Độ sâu sở thích: Bỏ trợ cho artist_diversity để phân biệt người nghe album vs người nghe playlist hỗn hợp.

Bảng 11: Feature Importance (Sắp xếp giảm dần)

2.5. Phân loại mức rủi ro

Xác suất churn của mỗi người dùng được quy đổi thành phần trăm (churn_risk_percent) và phân thành 3 mức rủi ro:

Mức rủi ro	Ngưỡng	Hành động khuyến nghị
HIGH	$\geq 70\%$	Ưu tiên can thiệp ngay
MEDIUM	40% - 69%	Gửi coupon, nội dung đặc biệt
LOW	$< 40\%$	Theo dõi định kỳ

Bảng 12: Phân loại mức rủi ro của bài toán dự đoán rời bỏ

Kết quả được lưu vào bảng default.churn_predictions với 3 cột: user_id, churn_risk_percent, risk_level.

3. Web Dashboard và Xuất Dữ liệu

File 03_dashboard_export.py thực hiện bước cuối cùng: tổng hợp tất cả kết quả từ các module trước và tạo ra bảng dữ liệu thống nhất phục vụ Dashboard:

- Đọc ba bảng: gold_user_features (gồm 7 feature), user_persona_results (nhóm người dùng) và churn_predictions (dự báo churn).

- Left Join tuần tự theo user_id: đảm bảo giữ lại toàn bộ người dùng trong Gold, kể cả những người chưa được phân cụm hoặc chưa có dự đoán churn.

- Xử lý NULL: Điền giá trị cho các trường thiếu (persona_name ="Chưa phân cụm", churn_risk_percent=0.0, risk_level="UNKNOWN").

- Xuất kết quả vào bảng `default.web_dashboard_data_v2` trên Databricks Unity Catalog, sẵn sàng kết nối với các BI Tools như Apache Superset, Power BI hoặc Tableau.

Kiến trúc dữ liệu Dashboard: Mỗi dòng trong bảng `web_dashboard_data_v2` đại diện cho một người dùng với đầy đủ thông tin: hành vi (7 features), nhóm persona, mức độ rủi ro churn và phần trăm nguy cơ cụ thể - đủ để xây dựng bất kỳ biểu đồ phân tích nào.

CHƯƠNG 9: ĐÁNH GIÁ VÀ KẾT QUẢ

1. Các chỉ số đánh giá mô hình

Module	Chỉ số đánh giá	Giá trị mục tiêu	Phương pháp
Phân cụm K-Means	Silhouette Score	~ 0.5	Spark MLlib ClusteringEvaluator (Squared Euclidean)
LightGCN RecSys	BPR Loss (huấn luyện)	Giảm dần qua epoch	MLflow metric logging theo từng epoch
Churn Prediction	AUC-ROC	> 0.80	BinaryClassificationEvaluator (areaUnderROC)
Data Pipeline	Data completeness	0 NULL trong Silver	DLT data quality rules

Bảng 13: Bảng chỉ số đánh giá mô hình

Metric(K=20)	BaseLightGCN	HybridTF-IDF($\alpha=0.25$)	Cải thiện
Recall@20	0.0405	0.0499	+23.2%
Precision@20	0.0301	0.0357	+18.6%
NDCG@20	0.0483	0.0597	+23.6%

Bảng 14: Kết quả của kiến trúc Hybrid so với thuật toán cơ sở LightGCN

2. Hạn chế và hướng phát triển trong tương lai

Hạn chế:

- Vấn đề cold-start và thiếu thông tin nội dung, Phụ thuộc vào dữ liệu lịch sử và giả định đồ thị tĩnh
- Không mô hình hóa tốt dữ liệu chuỗi thời gian, Khả năng tổng quát hóa hạn chế với hành vi mới

Hướng phát triển trong tương lai:

- Tích hợp dữ liệu âm thanh (Audio Features): Trích xuất đặc trưng âm thanh (tempo, key, energy) bằng librosa để cải thiện mô hình Content-based, đặc biệt hữu ích cho bài hát mới (Cold-Start).

- Session-based Recommendation: Áp dụng SASRec hoặc BERT4Rec để mô hình hóa chuỗi nghe nhạc trong phiên nghe, thay vì chỉ dùng trung bình đơn giản trong Late Fusion.
- Online Learning cho Churn Model: Cập nhật mô hình churn theo thời gian thực (streaming update) để bắt kịp thay đổi hành vi người dùng, thay vì batch retraining định kỳ.
- A/B Testing Framework: Xây dựng framework đánh giá mô hình gợi ý qua online metrics (Click-Through Rate, Stream Duration) thay vì chỉ offline metrics (Recall, NDCG).
- Graph Attention Network: Thay thế LightGCN bằng GAT (Graph Attention Network) để học được trọng số attention khác nhau cho từng lân cận, tăng khả năng biểu diễn.

TÀI LIỆU THAM KHẢO

- [1] He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. Proceedings of the 43rd International ACM SIGIR Conference, 639–648.
- [2] Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence, 452–461.
- [3] Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative Filtering for Implicit Feedback Datasets. Proceedings of the 8th IEEE International Conference on Data Mining, 263–272.
- [4] Wang, X., He, X., Wang, M., Feng, F., & Chua, T. S. (2019). Neural Graph Collaborative Filtering. Proceedings of the 42nd International ACM SIGIR Conference, 165–174.
- [5] Schedl, M., Zamani, H., Chen, C. W., Deldjoo, Y., & Elahi, M. (2018). Current Challenges and Visions in Music Recommender Systems Research. International Journal of Multimedia Information Retrieval, 7(2), 95–116.
- [6] Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). A profit driven data mining approach for customer churn prediction. European Journal of Operational Research, 218(1), 211–229.
- [7] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- [8] Xie, M., et al. (2016). Churn Prediction Using Temporal Behavioral Features in Music Streaming Services. Proceedings of the IEEE International Conference on Data Mining (ICDM).
- [9] Jain, A. K. (2010). Data clustering: 50 years beyond K-means. Pattern Recognition Letters, 31(8), 651–666.
- [10] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547.
- [11] Carbonell, J., & Goldstein, J. (1998). The Use of MMR for Diversity-Based Reranking. Proceedings of the 21st Annual International ACM SIGIR Conference, 335–336.

- [12] Zaharia, M., Xin, R. S., Wendell, P., et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56–65.
- [13] Zaharia, M., et al. (2018). MLflow: A Machine Learning Lifecycle Platform. *Proceedings of the NeurIPS Workshop on Systems for ML*.

Phân chia công việc

Doãn Đăng Khoa	- Xây dựng hệ gợi ý (LightGCN) - Xây dựng web demo - Viết báo cáo
Trịnh Đắc Trường	- Triển khai hệ thống lên azure, databrick (kiến trúc Lambda) - Dự đoán rời bỏ - Xử lý dữ liệu trên azure (các tầng)
Trần Văn Tuấn	- Phân cụm người dùng - Dự đoán rời bỏ - Literature Review
Lê Như Tùng	- Xây dựng hệ gợi ý (TF-IDF) - Xây dựng webdemo - Viết báo cáo